TALLER DE GO · 42 MÁLAGA · SESIÓN 2 · 12 DE NOVIEMBRE DE 2026

Bloque 2 · API HTTP

El `Store` del bloque 1 ya guarda en un fichero. En este bloque lo pones detrás de una API HTTP: seis rutas que reciben peticiones, llaman al `Store` y responden JSON.

Todo el trabajo está en `api.go`. Escribes cuatro handlers y dos funciones de ayuda. Los tests de `api_test.go` te dicen cuándo está bien cada uno, y al final la pruebas a mano con `curl`.

Lo que viene hecho

Las rutas

```
func NewHandler(store *Store) http.Handler {
    a := &api{store: store}
    mux := http.NewServeMux()
    mux.HandleFunc("GET /products", a.products)
    mux.HandleFunc("GET /products/{id}", a.product)
    mux.HandleFunc("POST /products/{id}", a.addProduct)
    mux.HandleFunc("POST /reserve", a.reserve)
    mux.HandleFunc("POST /reservations/{id}/confirm", a.confirm)
    mux.HandleFunc("POST /reservations/{id}/cancel", a.cancel)
    return mux
}
```

- `net/http` es de la biblioteca estándar: no hay framework.
- Cada ruta es un método HTTP, un path y la función que la atiende, que se llama handler.
- El `mux` reparte cada petición al handler de su ruta. Si ninguna coincide, responde él solo: 404, o 405 si el path existe con otro método.
- `{id}` es un comodín. El handler lee su valor con `r.PathValue("id")`.
- Los handlers son métodos de `api`, un struct que guarda el `Store`. Así cada handler llega a él con `a.store`.

RUTA	QUÉ HACE	RESPONDE
`GET /products`	Lista el catálogo	200 y un array de productos
`GET /products/{id}`	Un producto	200 y el producto
`POST /products/{id}`	Crea un producto	201 y el producto
`POST /reserve`	Crea una reserva	201 y la reserva
`POST /reservations/{id}/confirm`	Confirma una reserva	200 y la reserva
`POST /reservations/{id}/cancel`	Cancela una reserva	200 y la reserva

Dos handlers vienen escritos: `product`, que es tu modelo, y `addProduct`, que crea un producto y no es de ningún paso. Lo usa el programa de carga del bloque 3.

Un handler entero: `product`

```
func (a *api) product(w http.ResponseWriter, r *http.Request) {
    p, err := a.store.Product(r.PathValue("id"))   // leer la entrada y llamar al Store
    if err != nil {
        writeError(w, err)                         // si falla, responder el error
        return
    }
    writeJSON(w, http.StatusOK, p)                 // si no, responder el JSON
}
```

- Todos los handlers tienen esta firma. `r` es la petición: el path, las cabeceras y el cuerpo. `w` es donde se escribe la respuesta.
- Todos hacen lo mismo: leer la entrada, llamar al `Store`, y responder el error o el resultado.
- El `return` después de `writeError` importa: sin él se escribirían dos respuestas.
- El handler no sabe nada de SQL: solo habla con el `Store`.

Responder JSON: `writeJSON`

```
func writeJSON(w http.ResponseWriter, status int, v any) {
    w.Header().Set("Content-Type", "application/json")
    w.WriteHeader(status)
    json.NewEncoder(w).Encode(v)
}
```

- El orden es fijo: primero las cabeceras, después el código de estado y al final el cuerpo.
- `Encode` convierte en JSON lo que le pases: un struct, un slice o un mapa.
- Los errores se responden siempre con la misma forma, `{"error": "out of stock"}`. La da `errorBody("out of stock")`.

Leer JSON: `reserveRequest`

```
type reserveRequest struct {
    ProductID string `json:"product_id"`
    Quantity  int    `json:"quantity"`
}
```

Es el cuerpo de `POST /reserve`: `{"product_id": "p1", "quantity": 2}`. La etiqueta `json:"product_id"` dice cómo se llama cada campo en el JSON. Para llenar el struct con el cuerpo de la petición:

```
var req reserveRequest
err := json.NewDecoder(r.Body).Decode(&req)   // err != nil: el cuerpo no es JSON válido
```

Los errores y sus códigos

El `Store` devuelve los cinco errores de `reservation.go`. La API convierte cada uno en un código HTTP. Esa tabla la escribes tú en `writeError` (paso 2):

ERROR	CÓDIGO	POR QUÉ
`ErrProductNotFound`, `ErrReservationNotFound`	404	Lo que se pide no existe
`ErrOutOfStock`, `ErrIllegalTransition`	409	Una regla de la taquilla dice que no
`ErrInvalidQuantity`	400	La petición está mal
Cualquier otro	500	Ha fallado el servidor, no el cliente

Para saber qué error tienes se usa `errors.Is(err, ErrOutOfStock)`, nunca el texto del error.

El programa: `cmd/api`

```
store, err := reservations.OpenStore(*dbPath)                // tu Store del bloque 1
http.ListenAndServe(*addr, reservations.NewHandler(store))   // tu API, en un puerto
```

`cmd/api/main.go` viene entero. Abre el fichero `taquilla.db`, guarda dos productos si el fichero es nuevo y escucha en el puerto 8080. Los productos son `p1`, Ticket, con 10 unidades, y `p2`, Poster, con 3.

Los tests no arrancan ningún servidor: le pasan la petición al handler directamente y leen la respuesta, con `httptest`. Por eso van rápido y no necesitan puerto.

Paso a paso

Paso 1 · `products` (lo hacemos juntos)

```
go test -run TestAPI_products -v
```

- [ ] `products`: responde el catálogo entero con un 200.

Modelo: `product`, sin entrada que leer.

Paso 2 · `writeError`

```
go test -run TestAPI_product_unknown -v
```

- [ ] `writeError`: elige el código según el error, con la tabla de arriba, y responde el mensaje del error con `errorBody`.
- [ ] Añade `"errors"` a los `import` de `api.go`. No venía porque Go no compila con un import sin usar.

Un `switch` sin valor admite condiciones en cada `case`:

```
switch {
case errors.Is(err, ErrProductNotFound), errors.Is(err, ErrReservationNotFound):
    status = http.StatusNotFound
// ...
}
```

Paso 3 · `reserve` (el paso largo)

```
go test -run TestAPI_reserve -v
```

- [ ] `reserve`: lee el cuerpo de la petición, crea la reserva en el `Store` y la responde con un 201.
- [ ] Si el cuerpo no es JSON válido, responde un 400 sin llegar al `Store`.

Cómo se lee el cuerpo está más arriba, en "Leer JSON". El resto tiene la forma de `product`.

Hay dos 400 distintos. El del JSON roto lo decide el handler; el de la cantidad inválida viene del `Store` y pasa por `writeError`.

Paso 4 · `confirm`

```
go test -run TestAPI_confirm -v
```

- [ ] `respondWithReservation`: lee una reserva del `Store` y la responde con un 200.
- [ ] `confirm`: confirma la reserva cuyo `id` viene en la ruta y responde cómo ha quedado.

Modelo: `product`, que también lee el `id` de la ruta.

`Confirm` devuelve solo un error, no la reserva. Por eso el handler la vuelve a leer antes de responder.

Paso 5 · `cancel`

```
go test -run TestAPI_cancel -v
```

- [ ] `cancel`: igual que `confirm`, con el otro método del `Store`.

Comprobar el bloque entero

```
go test -run TestAPI -v
```

Has terminado cuando pasan los catorce `TestAPI_`. De `go test ./...` quedan fallando dos tests, los dos `TestConcurrent`: son el bloque 3.

Probarla a mano

En una terminal, arranca la API:

```
go run ./cmd/api
```

En otra, llámala:

```
curl localhost:8080/products
curl -X POST localhost:8080/reserve -d '{"product_id":"p1","quantity":2}'
curl -X POST localhost:8080/reservations/1/confirm
curl localhost:8080/products/p1
```

Y mira lo que hay en el fichero, con `sqlite3` o con la herramienta que uses:

```
sqlite3 -header -column taquilla.db 'select * from reservations'
```

En la tabla, `status` vale `1`: se guarda el número, y `Confirmed` es el 1. En el JSON viaja el nombre.

Para la API con Ctrl+C y arráncala otra vez: los datos siguen ahí. Para empezar de cero, borra `taquilla.db`.

Si algo falla

LO QUE VES	QUÉ PASA
`expected 200, got 501`	El handler sigue en `TODO`.
`expected 404, got 500`	`writeError` sigue en `TODO`, o le falta ese error en el `switch`.
`"errors" imported and not used`	Has añadido el import antes de usarlo: escribe `writeError`.
`address already in use`	Ya hay una API en ese puerto. Párala, o arranca con `-addr :8081`.